Department of Applied Mathematics and Statistics
COLORADO SCHOOL OF MINES
Math 550: Numerical Solutions of Partial Differential Equations
Charles Girard

Numerical PDEs Homework #2
Due Monday, November 2, 2026

Problem 1: Poisson Equation. Make sure the code from HW0 still works; use a finite dfference method to solve $\nabla^2 u = f(x, y)$ in the domain $x \in [0, 2\pi]$, $y \in [0, 2\pi]$ with Dirichelet boundary conditions on all sides. Choose f and boundary data so that the test solution

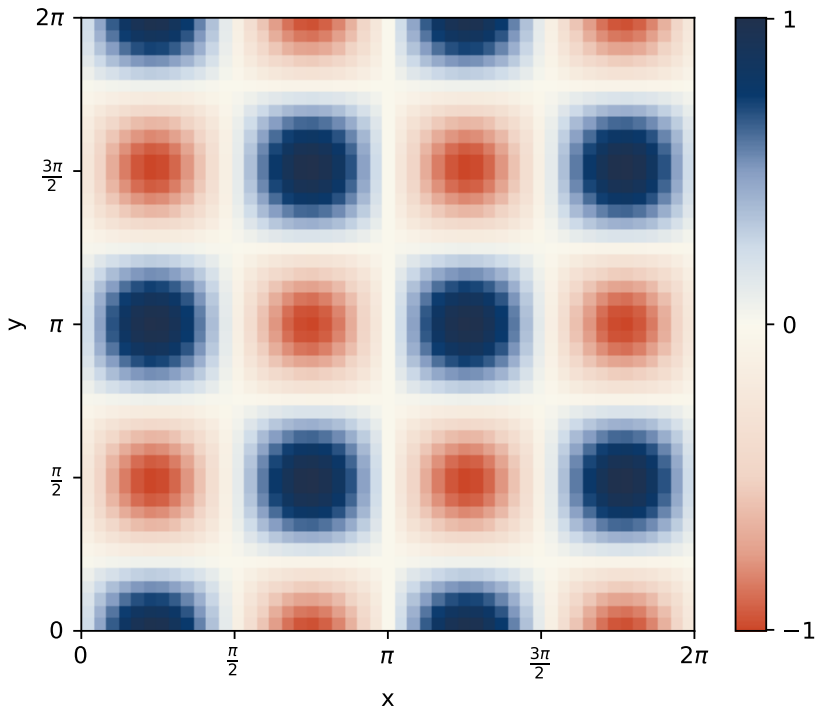
$$u(x, y) = \cos(2x)\sin(2y)$$

solves the problem. Then verify the solution converges in the second order.

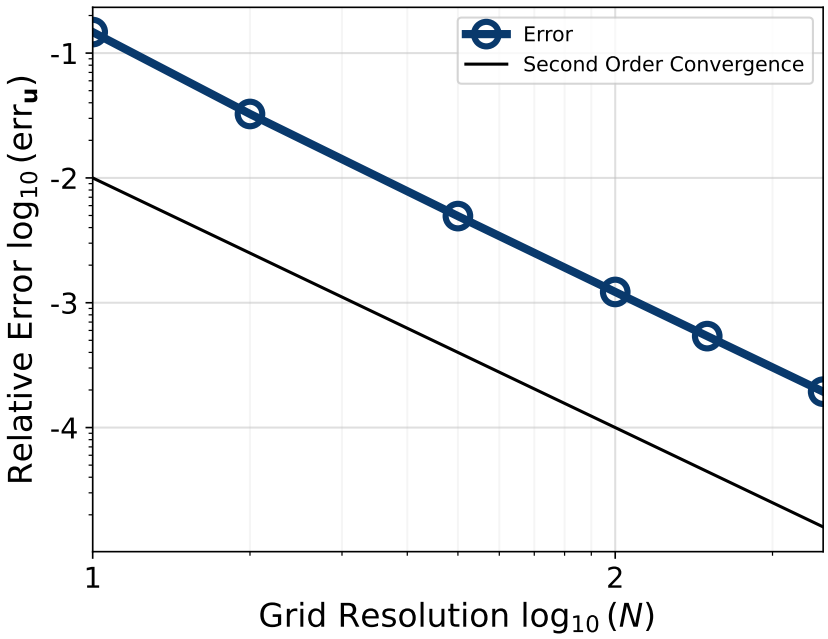
Result:
Solving for f gives

$$f(x, y) = -8\cos(2x)\sin(2y).$$

Using our code from HW0:



Approximate solution u_{apx}, with grid resolution $N = 50$.



Convergence plot for the Dirichlet problem.

Problem 2: Poisson Equation with an Immersed Boundary

Noe we'll consider an additional Dirichelet condition:

$$u(x^{ib}, y^{ib}) = \cos\big(2x^{ib}\big)\sin\big(2y^{ib}\big), \tag{1}$$

$$x^{ib} = \pi + \cos\theta, \quad y^{ib} = \pi + \sin\theta, \quad \theta \in [0, 2\pi]. \tag{2}$$

1. Discretize $\big(x^{ib}, y^{ib}\big)$ to have roughly the same immersed boundary resolution (point spacing) as the finite difference grid.

2. Create the spreading operator that maps the immersed boundary forces onto the finite difference grid:

$$\mathcal{S}^{ib}(q) = \sum_{k=1}^{N_{ib}} q_k \delta(x - x_k^{ib}, y - y_k^{ib}) \tag{3}$$

, where q_k are the forces exerted by the immersed boundary points, and δ_a is the 2D Gaussian with a variance $a = 1.2\Delta x$.

3. Discretizing our new PDE, we'll have N_{ib} more unknowns than equations. Therefore, we impose the dirichelet boundary condition:

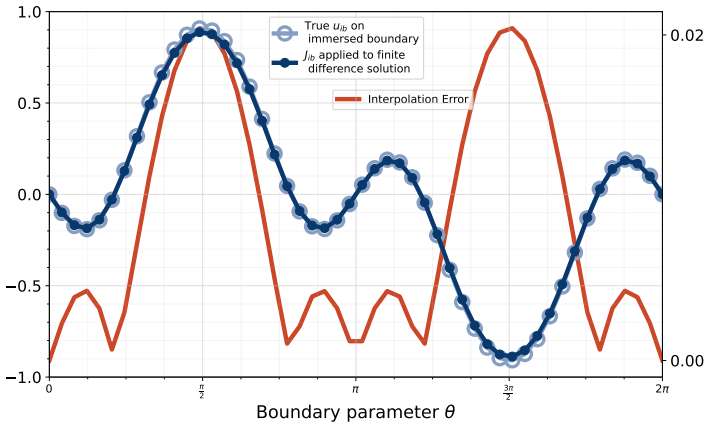
$$\int\int_{(x,y)} u(x,y)\delta_a(x - x_k^{ib}, y - y_k^{ib})\,dx\,dy = u(x_k^{ib}, y_k^{ib}), \quad k = 1, \ldots, N_{ib}, \tag{4}$$

approximated by a discrete sum:

$$\sum_{i,j} u_{i,j}\delta_a(x_i - x_k^{ib}, y_j - y_k^{ib})\Delta x^2 = u(x_k^{ib}, y_k^{ib}), \quad k = 1, \ldots, N_{ib}. \tag{5}$$

Call the operator on u as $\mathcal{J}^{ib}(u)$.

Result:
After creasing $\mathcal{S}^{ib}$ and $\mathcal{J}^{ib}$, we ensure that it interpolates the solution at the immersed boundary points correctly:



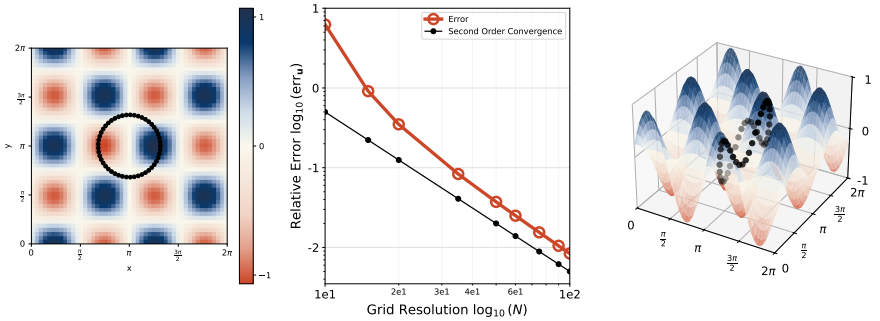
Interpolation error at the immersed boundary points.

Note that $\left(\mathcal{J}^{ib}\right)\top = \mathcal{S}^{ib}$. Therefore, we can use this relationship to instill confidence in $\mathcal{S}^{ib}$ given we're satisfied with $\mathcal{J}^{ib}$.

4. Discretize and solve the saddle point system,

$$\begin{bmatrix} \nabla^2 & -\mathcal{S}^{ib} \\ \mathcal{J}^{ib} & 0 \end{bmatrix} \begin{bmatrix} u(x,y) \\ q \end{bmatrix} = \begin{bmatrix} f(x,y) - \nabla^2_{\text{bdy}} u(x,y) \\ u(x_k^{ib}, y_k^{ib}) \end{bmatrix} \tag{6}$$

Result:



Complete solution with the immersed boundary.

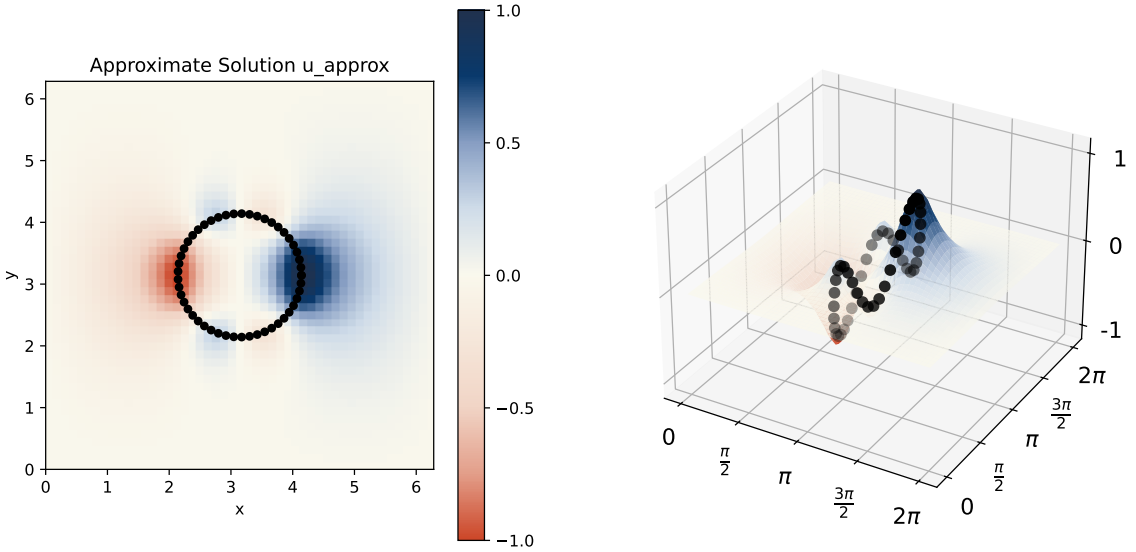
Problem 3: An almost minimal surface Not to solve a problem we don't know the answer to:

$$\nabla^2 u = \mathcal{S}^{ib}(q), \tag{7}$$

$$u(0, y) = u(2\pi, y) = u(x, 0) = u(x, 2\pi) = 0, \tag{8}$$

$$u(x_k^{ib}, y_k^{ib}) = \cos\left(2x^{ib}\right) \sin\left(2y^{ib}\right), x^{ib} = \pi + \cos\theta, \quad y^{ib} = \pi + \sin\theta, \quad \theta \in [0, 2\pi]. \tag{9}$$

Result:



Approximate solution for the almost minimal surface problem.

Problem 4: Charlie's Rule Play around with your cool code.

Result:

The reason this submission is so close to the very generous extended deadline is due to the over-ambitious tendencies of the author. That being said, please see the file in the repository titled `immersed_boundary.gif`

No further explanation or figures are provided here, so as not to spoil the surprise. Hopefully it doesn't disappoint.

-CG

(PS I was able to make it work more quickly with Schur decomposition! Thankfully the immersed boundary locations are unchanging, so the most time consuming part of making $\mathcal{S}^{ib}$ and $\mathcal{J}^{ib}$ is done only once.)